

Bagaimana pendekatan terbaik untuk mengelola perubahan dependensi dalam image berbasis Flask?

Menggunakan file `requirements.txt` dan RUN `pip install -r` dalam Dockerfile

Apa dampak jika urutan instruksi Dockerfile tidak efisien?

Build image menjadi lambat karena tidak memanfaatkan cache dengan baik

Bagaimana multi-stage builds membantu mengurangi ukuran image akhir?

Dengan memisahkan tahap build dan tahap runtime dalam image berbeda

Mengapa GitHub Actions menjadi pilihan populer untuk otomasi CI/CD berbasis container?

Karena terintegrasi langsung dengan GitHub dan dapat berjalan berdasarkan event seperti push

Mengapa proses monitoring container perlu diintegrasikan dalam sistem CI/CD modern?

Untuk memastikan aplikasi berjalan stabil dan mendeteksi error sejak awal

Mengapa proses enkripsi saat transfer image ke registry penting?

Untuk mencegah penyadapan data image selama proses transfer

Mengapa penting menjalankan langkah docker/build-push-action dalam workflow CI/CD?

Karena langkah ini melakukan build dan push image ke registry secara otomatis

Bagaimana Docker Compose mempermudah pengujian otomatis aplikasi?

Dengan menyusun environment pengujian terisolasi dan dapat direplikasi dengan cepat

Mengapa menyimpan data sensitif di Dockerfile merupakan praktik yang buruk?

karena data tersebut akan tercatat di layer image dan berpotensi bocor

Bagaimana GitHub Actions dapat mengamankan kredensial Docker Hub saat push image?

Menggunakan fitur Secrets pada GitHub repository

Mengapa `.dockerignore` sangat penting dalam proses build image?

Untuk mengecualikan file yang tidak relevan agar tidak ikut dibuild ke dalam image

Mengapa pendekatan modular dalam Docker Compose lebih disarankan?

Karena memungkinkan pengujian dan skalabilitas lebih baik dengan memisahkan fungsi tiap layanan

Mengapa penggunaan volume penting dalam Docker Compose?

Untuk menjaga data tetap ada meskipun kontainer dimatikan atau dihapus

Bagaimana algoritma round-robin pada load balancer bekerja?

Mendistribusikan permintaan secara bergiliran ke setiap container

Mengapa best practice menyarankan untuk tidak menyimpan data sensitif seperti token API di dalam Dockerfile? Apa alternatifnya?

Jangan simpan data sensitif di Dockerfile karena akan terekam di layer image dan berisiko bocor.

Jelaskan bagaimana penggunaan custom Docker image dapat meningkatkan efisiensi proses deployment aplikasi di lingkungan cloud!

Penggunaan custom Docker image meningkatkan efisiensi deployment karena seluruh dependensi dan konfigurasi aplikasi sudah terstandarisasi dalam satu image, sehingga mempercepat proses deploy dan mengurangi risiko ketidaksesuaian lingkungan.

IMAGE TO GIT

Langkah 1: Persiapan Repository

1. Buat repository baru di GitHub.
2. Clone repository ke komputer lokal:

```
git clone https://github.com/username/nama-repo.git
cd nama-repo
```

3. Tambahkan file berikut:
 - a. `app.py` (aplikasi python sederhana)
 - b. `requirements.txt` (misal: flask)
 - c. `Dockerfile`

Langkah 2: Tambahkan Secrets di GitHub

1. Buka repository GitHub.
2. Klik **Settings > Secrets and variables > Actions**.
3. Klik **New repository secret**:
 - a. `DOCKER_USERNAME`: isi dengan username Docker Hub.
 - b. `DOCKER_PASSWORD`: isi dengan token atau password Docker Hub.

Langkah 3: Buat Workflow GitHub Actions

1. Buat folder dan file baru:

```
mkdir -p .github/workflows
touch .github/workflows/docker-build.yml
```

2. Isi `docker-build.yml` dengan konfigurasi berikut:

```
name: Build and Push Docker Image
```

LOAD BALANCING]]

Load balancing adalah teknik distribusi lalu lintas jaringan ke beberapa server backend untuk meningkatkan keandalan dan performa sistem. Dalam lingkungan Docker, load balancing digunakan untuk mengelola banyak container aplikasi yang berjalan secara paralel.

Langkah-langkah Load Balancer dengan Docker & Nginx

1. **Persiapan Aplikasi**
 - Buat aplikasi sederhana (misalnya Flask) dalam `app.py`.
 - Buat `requirements.txt` dan Dockerfile untuk membangun image Python.
2. **Siapkan Docker Compose**
 - Buat `docker-compose.yml` dengan dua service container (`app1`, `app2`) dari image Flask.
 - Tambahkan service `nginx` sebagai load balancer.
3. **Konfigurasi Load Balancer (Nginx)**
 - Buat file `nginx.conf` untuk mendefinisikan `upstream` yang mengarah ke `app1:5000` dan `app2:5000`.
 - Arahkan trafik HTTP ke service-service tersebut menggunakan `proxy_pass`.
4. **Jalankan dan Uji**
 - Jalankan perintah `docker compose up --build`.
 - Akses `http://localhost` beberapa kali untuk melihat perubahan hostname, menandakan bahwa trafik didistribusikan ke container yang berbeda.